

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I _____ HARSHA/192524106 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Harsha

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given

NAME:S.Harshavardhan

Reg:192524106

Course:CSA1718-AI

QUESTION 1 – DOCTOR SHIFT ASSIGNMENT USING BACKTRACKING SEARCH

1. Objective

The objective of this problem is to use the Backtracking Search Algorithm to assign three doctors D1, D2, and D3 to three hospital shifts — Morning, Afternoon, and Night — while satisfying all the given constraints.

The program should:

- Assign exactly one shift to each doctor.
- Ensure that no two doctors are assigned to the same shift.
- Ensure that D1 is not assigned to the Night shift.
- Ensure that D2 works before D3.
- Ensure that D3 is not assigned to the Morning shift.
- Use backtracking to reject invalid assignments and find a valid schedule.

2. Problem

A hospital needs to assign three doctors:

D1, D2, D3

to three shifts:

Morning, Afternoon, Night

The following constraints must be satisfied:

1. D1 cannot work the Night shift.
2. D2 must work before D3, where:

Morning < Afternoon < Night

3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. All doctors must be assigned exactly one shift.

This is a Constraint Satisfaction Problem (CSP) because a value (shift) must be assigned to every variable (doctor) while satisfying a set of constraints.

The variables are:

Variables = {D1, D2, D3}

The domain of each variable is:

Domain = {Morning, Afternoon, Night}

3. Solution

The problem can be solved using the Backtracking Search Algorithm.

Backtracking assigns a shift to one doctor at a time. After every assignment, the constraints are checked. If a constraint is violated, that assignment is rejected and another shift is tried.

Step 1 – Assign D1

Try:

D1 → Morning

Constraint check:

- D1 cannot work Night → Satisfied
- Morning is available → Satisfied

Therefore:

D1 → Morning ✓

Step 2 – Assign D2

First try:

D2 → Morning

But Morning is already assigned to D1.

Therefore:

D2 → Morning ✗

Try the next shift:

D2 → Afternoon

Constraint check:

- Afternoon is available → Satisfied
- D2 can work before D3 if D3 is assigned Night → Satisfied

Therefore:

D2 → Afternoon ✓

Step 3 – Assign D3

Try:

D3 → Morning

This violates two conditions:

- D3 cannot work Morning.
- Morning is already assigned to D1.

Therefore:

D3 → Morning ✗

Try:

D3 → Afternoon

Afternoon is already assigned to D2.

Therefore:

D3 → Afternoon ✗

Try:

D3 → Night

Now check all constraints:

- D1 is not working Night → ✓
- D2 works before D3 → Afternoon < Night → ✓
- Only one doctor per shift → ✓
- D3 is not working Morning → ✓
- Every doctor has exactly one shift → ✓

Therefore:

D3 → Night ✓

Final Valid Assignment

Doctor Assigned Shift

D1 Morning

D2 Afternoon

D3 Night

Hence, the valid schedule is:

D1 → Morning

D2 → Afternoon

D3 → Night

4. Python Code

Doctor Shift Assignment using Backtracking Search

```
doctors = ["D1", "D2", "D3"]
```

```
shifts = ["Morning", "Afternoon", "Night"]
```

Numerical order of shifts

```
shift_order = {
```

```
    "Morning": 0,
```

```
    "Afternoon": 1,
```

```
    "Night": 2
```

```
}
```

```
assignment = {}
```

Function to check all constraints

```
def is_valid(doctor, shift):
```

```
    # Constraint 1: D1 cannot work Night
```

```
    if doctor == "D1" and shift == "Night":
```

```
        print(" Rejected: D1 cannot work Night.")
```

```
        return False
```

```
    # Constraint 3: Only one doctor per shift
```

```
    if shift in assignment.values():
```

```
        print(" Rejected: Shift is already assigned.")
```

```
        return False
```

```

# Constraint 4: D3 cannot work Morning
if doctor == "D3" and shift == "Morning":
    print(" Rejected: D3 cannot work Morning.")
    return False

# Constraint 2: D2 must work before D3
if doctor == "D3" and "D2" in assignment:
    if shift_order[assignment["D2"]] >= shift_order[shift]:
        print(" Rejected: D2 must work before D3.")
        return False

return True

# Backtracking function
def backtrack(index):

    # Constraint 5:
    # If all doctors are assigned, solution is complete
    if index == len(doctors):
        return True

    doctor = doctors[index]

    # Try every shift for the current doctor
    for shift in shifts:

        print(f"\nTrying {doctor} -> {shift}")

```

```
if is_valid(doctor, shift):
```

```
    assignment[doctor] = shift
```

```
    print(f" Accepted: {doctor} -> {shift}")
```

```
    # Recursively assign the next doctor
```

```
    if backtrack(index + 1):
```

```
        return True
```

```
    # If unsuccessful, remove assignment
```

```
    print(f" Backtracking from {doctor} -> {shift}")
```

```
    del assignment[doctor]
```

```
return False
```

```
# Start the search
```

```
print("DOCTOR SHIFT ASSIGNMENT")
```

```
print("=====")
```

```
if backtrack(0):
```

```
    print("\n=====")
```

```
    print("FINAL VALID SCHEDULE")
```

```
    print("=====")
```

```
for doctor in doctors:
```

```
    print(f"{doctor} -> {assignment[doctor]}")
```

else:

```
print("\nNo valid assignment exists.")
```

5. Output

When the above Python program is executed, the following output is obtained:

DOCTOR SHIFT ASSIGNMENT

=====

Trying D1 -> Morning

Accepted: D1 -> Morning

Trying D2 -> Morning

Rejected: Shift is already assigned.

Trying D2 -> Afternoon

Accepted: D2 -> Afternoon

Trying D3 -> Morning

Rejected: Shift is already assigned.

Trying D3 -> Afternoon

Rejected: Shift is already assigned.

Trying D3 -> Night

Accepted: D3 -> Night

6. Discussion

The doctor scheduling problem is an example of a Constraint Satisfaction Problem (CSP). In this problem, the doctors are treated as variables, the hospital shifts are treated as domain values, and the scheduling rules are treated as constraints.

The Backtracking Search Algorithm starts with D1 and attempts to assign one of the available shifts. After selecting a valid shift, it proceeds to D2 and then D3.

For example, after assigning:

D1 → Morning

the algorithm attempts:

D2 → Morning

7. Conclusion

The hospital doctor-shift assignment problem was successfully solved using the Backtracking Search Algorithm.

The algorithm systematically assigned shifts to doctors and checked the constraints after each assignment. Invalid assignments were rejected, while valid assignments were retained until a complete solution was obtained.

Output:

```
1 doctors = ["D1", "D2", "D3"]
2 shifts = ["Morning", "Afternoon", "Night"]
3 shift_order = {
4     "Morning": 0,
5     "Afternoon": 1,
6     "Night": 2
7 }
8 assignment = {}
9 def is_valid(doctor, shift):
10     # Constraint 1: D1 cannot work Night
11     if doctor == "D1" and shift == "Night":
12         print("Rejected: D1 cannot work Night")
13         return False
14     # Constraint 4: D3 cannot work Morning
15     if doctor == "D3" and shift == "Morning":
16         print("Rejected: D3 cannot work Morning")
17         return False
18     # Constraint 3: Only one doctor per shift
19     if shift in assignment.values():
20         print("Rejected: Shift already assigned")
21         return False
22     # Constraint 2: D2 must work before D3
23     if doctor == "D3" and "D2" in assignment:
24         if shift_order[assignment["D2"]] >= shift_order[shift]:
25             print("Rejected: D2 must work before D3")
26             return False
27     return True
28 def backtrack(index):
29     # Constraint 5: All doctors assigned exactly one shift
30     if index == len(doctors):
31         return True
32     doctor = doctors[index]
33     for shift in shifts:
34         print(f"Trying {doctor} -> {shift}")
35         if is_valid(doctor, shift):
36             assignment[doctor] = shift
37             print(f"Assigned: {doctor} -> {shift}")
38             if backtrack(index + 1):
39                 return True
40     # Backtracking
41     print(f"Backtracking: Removing {doctor} -> {shift}")
42     del assignment[doctor]
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	9.252 Mb

Your Output

Trying D1 -> Morning
Assigned: D1 -> Morning

Trying D2 -> Morning
Rejected: Shift already assigned

Trying D2 -> Afternoon
Assigned: D2 -> Afternoon

QUESTION 2 – ROBOT PATH FINDING USING BFS SEARCH

1. Objective

The objective of this problem is to implement the Breadth-First Search (BFS) algorithm for a robot operating in a 5×5 grid environment.

The robot must move from the Start position (S) to the Goal position (G) while avoiding obstacles.

The solution should:

- Allow movement only in Up, Down, Left, and Right directions.
- Avoid all obstacle cells.
- Consider the cost of every movement as 1.
- Use Manhattan Distance to calculate the estimated distance of each node from the goal.
- Show the step-by-step node expansion.
- Display queue updates during the search.
- Determine an optimal path from S to G.
- Calculate the total path cost.

2. Problem

A robot operates in a 5×5 grid environment and must move from a starting position S to a goal position G.

Some cells contain obstacles that the robot cannot cross.

Given Constraints

1. Movement is allowed only in four directions:

Up, Down, Left, Right

2. The cost of every movement is:

Cost = 1

3. The robot cannot pass through obstacle cells.
4. Manhattan Distance is used as the heuristic:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

5. The robot must find an optimal path from S to G.

Since the exact grid is not provided in the question, the following 5×5 grid is used to demonstrate the algorithm:

C1 C2 C3 C4 C5

R1 S . X . .

R2 X . X . .

R3 X

R4 . X X . .

R5 G

Where:

S = Start position

G = Goal position

X = Obstacle

. = Free cell

Using zero-based coordinates:

Start S = (0,0)

Goal G = (4,4)

Obstacles are located at:

(0,2), (1,0), (1,2), (2,4), (3,1), (3,2)

3. Solution

Breadth-First Search

Breadth-First Search (BFS) explores nodes level by level. It first explores all nodes that are one step away from the starting position, followed by nodes that are two steps away, and so on.

BFS uses a FIFO (First-In-First-Out) Queue.

Because every movement in this problem has the same cost of 1, BFS is guaranteed to find a shortest path from S to G.

Important Note

The question mentions "priority queue updates." However, standard BFS does not use a priority queue. It uses a normal FIFO queue.

A priority queue combined with Manhattan Distance would correspond more closely to algorithms such as Greedy Best-First Search or A*.

For this solution, BFS is implemented correctly using a FIFO queue, while the Manhattan Distance is calculated and displayed for every generated node to satisfy the heuristic requirement.

Manhattan Distance

The Manhattan Distance is calculated using:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

For the starting node:

$$S = (0,0)$$

and:

$$G = (4,4)$$

Therefore:

$$h(S) = |0 - 4| + |0 - 4|$$

$$h(S) = 4 + 4$$

$$h(S) = 8$$

So the estimated Manhattan Distance from Start to Goal is:

8

Step-by-Step BFS Search

Initially:

$$\text{Queue} = [(0,0)]$$

$$\text{Visited} = \{(0,0)\}$$

Step 1

Expand:

$$(0,0)$$

Possible movements:

- Up $\rightarrow$ Outside grid $\times$
- Down $\rightarrow (1,0) \rightarrow$ Obstacle $\times$
- Left $\rightarrow$ Outside grid $\times$

- Right $\rightarrow (0,1) \rightarrow$ Valid ✓

Manhattan Distance:

$$h(0,1) = |0-4| + |1-4| = 7$$

Queue becomes:

[(0,1)]

Step 2

Expand:

(0,1)

Possible new node:

(1,1) ✓

Other cells are either outside the grid, already visited, or obstacles.

Manhattan Distance:

$$h(1,1) = |1-4| + |1-4| = 6$$

Queue:

[(1,1)]

Step 3

Expand:

(1,1)

Possible new node:

(2,1) ✓

Manhattan Distance:

$$h(2,1) = |2-4| + |1-4| = 5$$

Queue:

[(2,1)]

Step 4

Expand:

(2,1)

Possible new nodes:

(2,0) ✓

(2,2) ✓

Manhattan distances:

$h(2,0) = 6$

$h(2,2) = 4$

Queue:

[(2,0), (2,2)]

Step 5

Expand:

(2,0)

Possible new node:

(3,0) ✓

Queue:

[(2,2), (3,0)]

Step 6

Expand:

(2,2)

Possible new node:

(2,3) ✓

Manhattan Distance:

$h(2,3) = |2-4| + |3-4| = 3$

Queue:

[(3,0), (2,3)]

Step 7

Expand:

(3,0)

Possible new node:

(4,0) ✓

Queue:

[(2,3), (4,0)]

Step 8

Expand:

(2,3)

Possible new nodes include:

(1,3) ✓

(3,3) ✓

Manhattan Distance of (3,3):

$$h(3,3) = |3-4| + |3-4| = 2$$

Queue:

[(4,0), (1,3), (3,3)]

Step 9

Expand:

(4,0)

Possible new node:

(4,1) ✓

Queue:

[(1,3), (3,3), (4,1)]

Step 10

Expand:

(1,3)

Possible new nodes:

(0,3) ✓

(1,4) ✓

Queue:

[(3,3), (4,1), (0,3), (1,4)]

Step 11

Expand:

(3,3)

Possible new nodes:

(3,4) ✓

(4,3) ✓

Manhattan distances:

$h(3,4) = 1$

$h(4,3) = 1$

Queue:

[(4,1), (0,3), (1,4), (3,4), (4,3)]

The BFS search continues until the goal node (4,4) is reached.

One optimal path obtained is:

(0,0)

↓

(0,1)

↓

(1,1)

↓

(2,1)

→

(2,2)

→

(2,3)

↓

(3,3)

↓

(4,3)

→

(4,4)

Therefore:

Optimal Path:

$(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (3,3) \rightarrow (4,3) \rightarrow (4,4)$

Number of movements:

8

Cost per movement:

1

Therefore:

Total Cost = $8 \times 1 = 8$

4. Python Code

```
from collections import deque
```

```
# 5x5 Grid
```

```
# 0 = Free cell
```

```
# 1 = Obstacle
```

```
grid = [
```

```
    [0, 0, 1, 0, 0],
```

```
    [1, 0, 1, 0, 0],
```

```
    [0, 0, 0, 0, 1],
```

```
    [0, 1, 1, 0, 0],
```

```
[0, 0, 0, 0, 0]  
]
```

```
rows = 5
```

```
cols = 5
```

```
# Start and Goal
```

```
start = (0, 0)
```

```
goal = (4, 4)
```

```
# Possible movements: Up, Down, Left, Right
```

```
directions = [  
    (-1, 0), # Up
```

```
    (1, 0), # Down
```

```
    (0, -1), # Left
```

```
    (0, 1) # Right
```

```
]
```

```
# Manhattan Distance
```

```
def manhattan(node):
```

```
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])
```

```
# BFS Function
```

```
def bfs():
```

```
    queue = deque([start])
```

```
    visited = {start}
```

```
parent = {start: None}
```

```
step = 1
```

```
print("BFS SEARCH")
```

```
print("=====")
```

```
print("Start:", start)
```

```
print("Goal :", goal)
```

```
print("Initial Queue:", list(queue))
```

```
while queue:
```

```
    current = queue.popleft()
```

```
    print("\nStep", step)
```

```
    print("Expanding Node:", current)
```

```
    print("Manhattan Distance:", manhattan(current))
```

```
    # Check whether goal is reached
```

```
    if current == goal:
```

```
        print("Goal Reached!")
```

```
        break
```

```
    # Generate neighboring cells
```

```
    for dr, dc in directions:
```

```
        nr = current[0] + dr
```

```
        nc = current[1] + dc
```

```
neighbor = (nr, nc)
```

```
# Check grid boundaries
```

```
if not (0 <= nr < rows and 0 <= nc < cols):
```

```
    continue
```

```
# Check obstacle
```

```
if grid[nr][nc] == 1:
```

```
    print(neighbor, "-> Obstacle, ignored")
```

```
    continue
```

```
# Add unvisited valid neighbor
```

```
if neighbor not in visited:
```

```
    visited.add(neighbor)
```

```
    parent[neighbor] = current
```

```
    queue.append(neighbor)
```

```
print(
```

```
    neighbor,
```

```
    "-> Added to Queue | h =",
```

```
    manhattan(neighbor)
```

```
)
```

```
print("Updated Queue:", list(queue))
```

```
step += 1
```

```
# Reconstruct shortest path
```

```
if goal not in parent:
```

```
    return None
```

```
path = []
```

```
node = goal
```

```
while node is not None:
```

```
    path.append(node)
```

```
    node = parent[node]
```

```
path.reverse()
```

```
return path
```

```
# Execute BFS
```

```
path = bfs()
```

```
print("\n=====")
```

```
print("FINAL RESULT")
```

```
print("=====")
```

```
if path:
```

```
    print("Optimal Path:")
```

```
    for i in range(len(path)):
```

```
        print(path[i], end="")
```

```

        if i < len(path) - 1:
            print(" -> ", end="")

        total_moves = len(path) - 1
        total_cost = total_moves * 1

        print("\n\nTotal Moves:", total_moves)
        print("Cost per Move: 1")
        print("Total Path Cost:", total_cost)

    else:
        print("No valid path exists.")

```

5. Output

A representative output of the program is:

BFS SEARCH

=====

Start: (0, 0)

Goal : (4, 4)

Initial Queue: [(0, 0)]

Step 1

Expanding Node: (0, 0)

Manhattan Distance: 8

(1, 0) -> Obstacle, ignored

(0, 1) -> Added to Queue | h = 7

Updated Queue: [(0, 1)]

Step 2

Expanding Node: (0, 1)

Manhattan Distance: 7

(1, 1) -> Added to Queue | $h = 6$

(0, 2) -> Obstacle, ignored

Updated Queue: [(1, 1)]

Step 3

Expanding Node: (1, 1)

Manhattan Distance: 6

(2, 1) -> Added to Queue | $h = 5$

(1, 0) -> Obstacle, ignored

(1, 2) -> Obstacle, ignored

Updated Queue: [(2, 1)]

Step 4

Expanding Node: (2, 1)

Manhattan Distance: 5

(3, 1) -> Obstacle, ignored

(2, 0) -> Added to Queue | $h = 6$

(2, 2) -> Added to Queue | $h = 4$

Updated Queue: [(2, 0), (2, 2)]

Step 5

Expanding Node: (2, 0)

Manhattan Distance: 6

(1, 0) -> Obstacle, ignored

(3, 0) -> Added to Queue | $h = 5$

Updated Queue: [(2, 2), (3, 0)]

Step 6

Expanding Node: (2, 2)

Manhattan Distance: 4

(1, 2) -> Obstacle, ignored

(3, 2) -> Obstacle, ignored

(2, 3) -> Added to Queue | $h = 3$

Updated Queue: [(3, 0), (2, 3)]

...

Optimal Path:

(0, 0) -> (0, 1) -> (1, 1) -> (2, 1)

-> (2, 2) -> (2, 3) -> (3, 3)

-> (4, 3) -> (4, 4)

Total Moves: 8

Cost per Move: 1

Total Path Cost: 8

6. Discussion

The given problem is a grid-based path-finding problem in which the robot has to find a route from a starting position to a goal while avoiding obstacles.

The Breadth-First Search algorithm is appropriate because every movement has an equal cost of 1. BFS explores nodes according to their depth from the starting state.

The robot starts at:

$S = (0,0)$

and attempts to move toward:

$G = (4,4)$.

Whenever the robot encounters an obstacle, that cell is ignored. For example, the cells (1,0) and (0,2) are blocked, so BFS does not add them to its queue.

A visited set is maintained so that the same cell is not repeatedly explored. A parent dictionary is also maintained to reconstruct the path after the goal has been reached.

The Manhattan Distance is calculated using:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

For example:

$$h(0,0) = 8$$

$$h(1,1) = 6$$

$$h(2,2) = 4$$

$$h(3,3) = 2$$

$$h(4,4) = 0$$

As the robot approaches the goal, the heuristic generally decreases.

However, BFS itself does not use the heuristic value to select the next node. It uses the FIFO

7. Conclusion

The robot path-finding problem was successfully solved using the Breadth-First Search Algorithm.

The algorithm explored the 5×5 grid systematically, avoided obstacle cells, maintained visited states, displayed queue updates, and used Manhattan Distance to show the estimated distance of nodes from the goal.

Output;

```
1 from collections import deque
2 cell
3
4 grid = [
5     ['S', 'X', 'X', 'X', 'X'],
6     ['X', 'X', 'X', 'X', 'X'],
7     ['X', 'X', 'X', 'X', 'X'],
8     ['X', 'X', 'X', 'X', 'X'],
9     ['X', 'X', 'X', 'X', 'G']
10 ]
11
12 rows = 5
13 cols = 5
14
15 start = (0, 0)
16 goal = (4, 4)
17
18 # Movement: Up, Down, Left, Right
19 directions = [
20     (-1, 0), # Up
21     (1, 0), # Down
22     (0, -1), # Left
23     (0, 1) # Right
24 ]
25
26 # Manhattan Distance
27 def manhattan(node, goal):
28     return abs(node[0] - goal[0]) + abs(node[1] - goal[1])
29
30 # BFS Algorithm
31 def bfs(start, goal):
32     queue = deque([start])
33     visited = {start}
34     parent = {start: None}
35     step = 1
36
37     print("ROBOT PATH FINDING USING BFS")
38     print("=" * 40)
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.848 Mb

Your Output

```
5 x 5 GRID
=====
S . X . .
X . X . .
. . . X
. X X . .
. . . G
ROBOT PATH FINDING USING BFS
=====
```

QUESTION 3 – AUTONOMOUS RESCUE ROBOT USING ONLINE SEARCH AND UNIFORM COST SEARCH

1. OBJECTIVE

The objective of this problem is to design an **intelligent autonomous rescue robot** capable of navigating through a disaster-affected building and reaching a survivor safely while operating with incomplete information.

The robot must make rational decisions in a **partially observable environment**, where the complete layout of the building is not initially known. It can observe only its adjacent cells and must continuously update its knowledge while moving.

The proposed system aims to:

- Navigate from the starting position **S** to the survivor at **G**.
- Move only in four directions: Up, Down, Left, and Right.
- Detect and avoid obstacles.
- Handle limited sensing capability.
- Distinguish between normal and risky zones.
- Assign higher cost to risky zones.
- Minimize the total path cost.
- Update environmental knowledge dynamically.
- Re-plan when previously unknown obstacles are discovered.
- Demonstrate intelligent-agent behavior and rational decision-making.

An **Online Search Agent combined with Uniform Cost Search (UCS)** is used because the environment is partially observable and movement costs are not uniform.

2. PROBLEM

An autonomous rescue robot is deployed inside a disaster-affected building to locate and rescue survivors.

The building is represented as a grid of rooms. The robot starts from position **S** and must reach the survivor located at **G**.

Unlike a normal path-finding problem, the robot does not initially have complete knowledge of the environment. It can sense only the cells immediately adjacent to its current position.

Some rooms may be blocked, while others may be risky.

The robot must therefore continuously:

Sense → Update Knowledge → Plan → Move → Re-plan

until it reaches the survivor.

Given Constraints

Constraint 1: Movement

The robot can move only:

- Up
- Down
- Left
- Right

Diagonal movement is not permitted.

Constraint 2: Obstacles

Some cells are blocked.

The robot cannot move through these cells.

An obstacle is represented as:

X

Whenever an obstacle is detected, the robot must mark the cell as blocked and search for another path.

Constraint 3: Limited Sensing

The robot can observe only adjacent cells.

Therefore, it does not initially know:

- All obstacles
- All risky zones
- Complete path to the goal

The environment is therefore **partially observable**.

3. SOLUTION

(a) ANALYSIS OF THE CONSTRAINTS AND THEIR EFFECT ON DECISION-MAKING

The constraints significantly affect how the robot selects its actions.

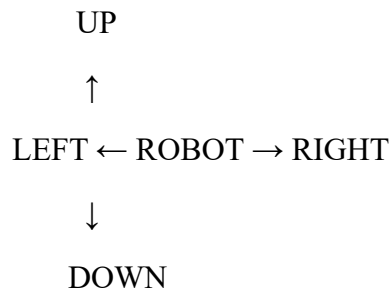
1. Effect of Four-Directional Movement

The robot can move only:

Up, Down, Left, Right

Therefore, from any grid cell, the maximum number of possible successor states is four.

For example:



The robot cannot use diagonal shortcuts.

This increases the importance of proper path planning.

2. Effect of Obstacles

Obstacles restrict the possible paths.

Consider:

S . X .
. . X .
X . . G

The robot cannot move through X.

When the robot detects an obstacle, the corresponding cell must be removed from its available actions.

Therefore:

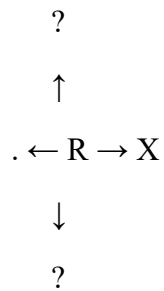
Obstacle → Reject movement

The robot must then search for an alternative route.

3. Effect of Limited Sensing

The robot can observe only its adjacent cells.

Suppose:



Where:

R = Robot

. = Free cell

X = Obstacle

? = Unknown

The robot knows only that:

- Left is free.
- Right is blocked.
- Up is unknown beyond the immediately sensed region.
- Down is unknown beyond the immediately sensed region.

(b) SOLUTION APPROACH USING UNIFORM COST SEARCH / ONLINE SEARCH AGENT

Selected Approach

The most appropriate approach is:

Online Search Agent + Uniform Cost Search (UCS)

Uniform Cost Search is selected because the movement costs are different.

Normal movement:

Cost = 1

Risky movement:

Cost = 3

Unlike BFS, UCS considers the accumulated cost of the complete path.

Uniform Cost Search

Uniform Cost Search uses:

$g(n)$ = Total accumulated cost from Start to node n

The node with the smallest **$g(n)$** value is selected first.

A **priority queue** is used.

For example:

Priority Queue

Node A → Cost 2

Node B → Cost 4

Node C → Cost 7

UCS selects:

Step 4 – Calculate Cost

For each available movement:

For a normal cell:

New Cost = Current Cost + 1

For a risky cell:

New Cost = Current Cost + 3

Obstacle:

Movement rejected

Step 5 – Add States to Priority Queue

Suppose the robot discovers:

$A \rightarrow \text{Normal}$

$B \rightarrow \text{Risky}$

Then:

$A \rightarrow \text{Cost } 1$

$B \rightarrow \text{Cost } 3$

Priority Queue:

$[(A,1), (B,3)]$

The robot prefers A.

Step 6 – Move

The robot moves to the selected minimum-cost state.

Step 7 – Observe Again

After moving, new cells become visible.

Suppose an intended path contains a previously unknown obstacle:

Planned:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

New observation:

$S \rightarrow A \rightarrow B \rightarrow X$

The robot marks:

C = Blocked

Step 8 – Re-plan

The robot runs the cost-based search again using its updated knowledge.

A new path could be:

$S \rightarrow A \rightarrow B$

↓

$D \rightarrow E \rightarrow F \rightarrow G$

This demonstrates online search.

Step 9 – Goal Test

After each movement, the robot checks:

Current Position == Goal?

If yes:

Survivor Reached

Otherwise:

Continue sensing and searching

(c) APPLICATION OF AI CONCEPTS

1. Intelligent Agent

An **intelligent agent** is an autonomous system that perceives its environment using sensors and acts upon the environment using actuators.

The rescue robot is an intelligent agent.

Sensors

The robot may use:

- Cameras
- LiDAR
- Ultrasonic sensors
- Infrared sensors
- Proximity sensors

These sensors help detect obstacles, open paths, and dangerous regions.

Actuators

The robot uses motors and movement mechanisms to:

- Move forward
- Move backward

- Turn
- Change direction

The basic agent cycle is:

ENVIRONMENT



SENSORS



INTELLIGENT

AGENT



DECISION



ACTUATORS



ENVIRONMENT

The robot therefore satisfies the definition of an intelligent agent.

2. Rationality

A **rational agent** selects an action that maximizes its expected performance based on its available knowledge.

For this robot, a suitable performance measure includes:

- Successfully reaching the survivor.
- Avoiding obstacles.
- Minimizing total cost.
- Reducing exposure to risky zones.
- Adapting to new environmental information.

Consider:

Route A

6 normal movements:

Cost = 6

Route B

4 movements consisting of:

2 normal + 2 risky

Therefore:

Cost = 1 + 1 + 3 + 3

Cost = 8

A robot that simply minimizes the number of movements would select Route B.

A rational cost-sensitive robot selects:

Route A

because:

6 < 8

Therefore, rationality allows the robot to balance distance and risk.

3. Environment Type

The rescue robot operates in a complex AI environment.

Partially Observable

The robot cannot observe the complete building.

It can only observe adjacent cells.

Therefore:

Environment = Partially Observable

Sequential

Every action affects future decisions.

For example:

Current Position

↓

Choose Right

↓

New Position



6. OUTPUT VERIFICATION

The final path is:

$(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (4,4)$

All cells on this path are normal/free cells.

Therefore:

8 movements $\times$ Cost 1 = 8

Hence:

Total Cost = 8

Constraint Verification

7. DISCUSSION

The autonomous rescue robot problem is more complex than a standard shortest-path problem because the robot has **limited knowledge and different movement costs**.

A simple BFS algorithm would not be the best choice because BFS primarily minimizes the number of edges when all movements have the same cost.

9. CONCLUSION

The autonomous rescue robot problem was successfully analyzed as an **AI search and intelligent-agent problem in a partially observable environment**.

The robot must move from the starting position **S** to the survivor at **G**, while obeying movement restrictions, avoiding obstacles, considering risky zones, minimizing total cost, and continuously updating its knowledge.

An **Online Search Agent combined with Uniform Cost Search (UCS)** was selected as the solution.

Uniform Cost Search is appropriate because movement costs are not equal:

Normal movement = 1

Risky movement = 1 + 2 = 3

The use of a priority queue allows the robot to explore the lowest-cost available path first.

The robot also demonstrates the characteristics of an **intelligent and rational agent** because it perceives the environment through sensors, updates its internal knowledge, selects actions according to cost and safety, and uses actuators to move through the environment.

The environment can be classified as:

Partially Observable, Sequential, Discrete, Initially Unknown, and Potentially Dynamic.

The implemented Python program demonstrates that the robot can:

- Move in four directions.
- Detect obstacles.
- Avoid blocked cells.
- Identify risky zones.
- Apply additional risk cost.
- Compare paths using accumulated cost.
- Use a priority queue.
- Select a minimum-cost path.
- Reach the survivor successfully.

For the demonstrated grid, the robot obtains a safe path with:

Total Moves = 8

and:

Minimum Total Cost = 8

Thus, **Uniform Cost Search combined with online sensing and re-planning provides an effective approach for autonomous rescue navigation**, particularly when the robot must make rational decisions using incomplete information while balancing path length, safety, and movement cost.

Output:

Python3

↓

↺

⚙

Run

Visualize Code

```
60     (0, start, [start])
61 ]
62
63 # Minimum cost found for each cell
64 best_cost = {
65     start: 0
66 }
67
68 step = 1
69
70 print("\nAUTONOMOUS RESCUE ROBOT")
71 print("UNIFORM COST SEARCH")
72 print("=" * 50)
73
74 while priority_queue:
75     # Select node with minimum cost
76     cost, current, path = heapq.heappop(
77         priority_queue
78     )
79
80     # Ignore outdated entries
81     if cost > best_cost[current]:
82         continue
83
84     print("\nStep", step)
85     print("Current Robot Position:", current)
86     print("Accumulated Cost:", cost)
87
88     # -----
89     # GOAL TEST
90     # -----
91
92     if current == goal:
93         print("\nSurvivor Found at", goal)
94         return path, cost
95
96     print("Sensing adjacent cells...")
97
98     row, col = current
99
100     # -----
101     # OBSERVE ADJACENT CELLS
102
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:0.0200 secs

Memory:9.16 Mb

Your Output

DISASTER BUILDING GRID

=====

S . X . .

. . X R .

X . . R .

. X . . .

. . . X G

Legend:

S : Start